

AI-Based Social Issue Detection

(AI Incident Manager) — Full Project Report

Project Title: AI-Based Social Issue Detection (AI Incident Manager) **Type:** Full-Stack Web Application with AI/NLP Integration **Domain:** Natural Language Processing · Public Safety · Civic Technology

1. Abstract

This project is an **AI-powered Incident Management System** that automatically ingests, analyzes, and prioritizes social media posts and user-submitted text related to civic issues, emergencies, online harassment, and mental health crises. By combining **Natural Language Processing (NLP)** models served via Hugging Face's Inference API with a custom **rule-based risk engine**, the system can determine:

1. **What** the post is about (Topic Classification)
2. **How** the author is feeling (Emotion Detection)
3. **Whether** the language is genuine or hyperbolic (Context Validation)
4. **How urgently** authorities should respond (Severity/Priority Scoring)

The application provides an admin-facing dashboard where processed incidents are displayed with full AI verdicts, enabling rapid triage and resolution of real-world social issues at scale.

2. Problem Statement

Social media platforms generate millions of posts daily — many of which report real emergencies, civic complaints, harassment, or mental health crises. Manual monitoring is impossible at scale, and simple keyword-matching systems produce unacceptable false-

positive rates (e.g., flagging *"This traffic makes me want to jump off a bridge"* as a suicide threat).

This project solves three key problems:

Problem	How It's Solved
Volume — Too many posts for humans to read	Automated AI pipeline processes posts in bulk (up to 50 per batch)
Nuance — Keyword matching can't distinguish sarcasm, hyperbole, or context	Zero-shot NLP models understand meaning, not just words
Prioritization — Not all issues are equally urgent	A 10-rule severity engine assigns Critical / High / Medium / Low priority

3. Technology Stack

3.1 Frontend

Technology	Version	Purpose
React.js	19.x	Component-based UI framework
Vite	8.x	Lightning-fast build tool and dev server
React Router DOM	7.x	Client-side routing and navigation
Axios	1.x	HTTP client for API communication
Vanilla CSS	—	Custom styling with modern design patterns

3.2 Backend

Technology	Version	Purpose
Node.js	—	JavaScript runtime for the server
Express.js	4.x	RESTful API framework
Mongoose	7.x	MongoDB ODM for data modeling
JSON Web Tokens (JWT)	9.x	Stateless authentication

bcrypt.js	3.x	Password hashing (salted, 10 rounds)
Helmet	7.x	HTTP security headers
Morgan	1.x	HTTP request logging
Cookie-Parser	1.x	JWT cookie extraction
Express-Validator	7.x	Input validation
dotenv	16.x	Environment variable management
Nodemon	3.x	Hot-reloading dev server

3.3 Database

Technology	Purpose
MongoDB (Local, port 27017)	NoSQL document database for storing posts, users, and analysis results

3.4 AI / Machine Learning

Model	Provider	Task
facebook/bart-large-mnli	Hugging Face Inference API	Zero-Shot Classification (Topic + Filters)
j-hartmann/emotion-english-distilroberta-base	Hugging Face Inference API	Emotion Detection (7 emotions)

3.5 Ports Configuration

Service	Port
Frontend (Vite)	5173
Backend (Express)	5001
MongoDB	27017

4. System Architecture

The system follows a **three-tier architecture** with an integrated AI pipeline:



5. The 4-Phase AI Analysis Pipeline

Every post submitted to the system — whether individually or via bulk dataset upload — passes through a master orchestrator (`analysisService.js`) that executes four sequential phases.

Phase 1 — Text Preprocessing

File: `utils/preprocessor.js`

Step	Operation	Example
1	Lowercase conversion	"HELP ME!!!" → "help me!!!"
2	URL removal	Strips <code>https://...</code> links
3	Punctuation & emoji removal	"help me!!!" → "help me"
4	Whitespace normalization	Collapses multiple spaces to single
5	Stop-word removal & tokenization	"help me now" → ["help", "now"]

Output: `{ cleanedText: string, tokens: string[] }`

Phase 2 — Dual Engine Assessment (Parallel Execution)

Two AI engines run **concurrently** via `Promise.all()` to minimize latency:

Engine A: Topic Classifier

File: `ai-engine/engines/topicClassifier.js` Model: `facebook/bart-large-mnli` (Zero-Shot Classification)

Classifies the post into one of **6 predefined categories**:

Category	Example Post
Civic Issue	"The roads in sector 12 have been broken for 3 months"
Crime / Violence	"There was a stabbing near the park last night"

Self-Harm / Depression	"I don't want to live anymore, nothing matters"
Cyberbullying / Harassment	"Someone keeps sending me threatening messages"
Emergency / Disaster	"Building collapsed on main street, people trapped"
General Complaint	"The food delivery was 2 hours late again"

- **Confidence threshold:** If the model's top score is below 0.35, the post defaults to **General Complaint**.
- **Fallback:** If the Hugging Face API is unavailable, a keyword dictionary with 100+ terms is used.

Output: `{ primaryTopic: string, topicConfidence: number (0.0–1.0) }`

Engine B: Emotion Scorer

File: `ai-engine/engines/emotionScorer.js` **Model:** `j-hartmann/emotion-english-distilroberta-base`

Detects the intensity of **8 emotions**:

Emotion	Source	Notes
Anger	Direct from model	—
Fear	Direct from model	—
Sadness	Direct from model	—
Despair	Derived (sadness + context amplifiers)	Critical for self-harm detection
Disgust	Direct from model	Replaces older "annoyance" field
Joy	Direct from model	—
Surprise	Direct from model	—
Neutral	Direct from model	—

Despair Derivation Logic: If sadness is detected AND the text contains phrases from a curated **DESPAIR_AMPLIFIERS** list (e.g., "don't want to live", "end it all", "hopeless", "kill myself"), the despair score is amplified. This is critical because the `sadness` label alone is too generic — **despair** is a more specific indicator of self-harm risk.

Volatility Flag: If **2 or more** emotions exceed the 0.65 threshold simultaneously (e.g., fear: 0.90, anger: 0.88), the post is flagged as **emotionally volatile** — a key escalation factor in Phase 4.

Output: { emotions: {8 scores}, dominantEmotion, dominantEmotionScore, volatilityFlag, analysisMethod }

Phase 3 — Context Validation (Filter Layer)

File: `ai-engine/filters/filterLayer.js` **Model:** `facebook/bart-large-mnli` (Zero-Shot Classification, reused)

This layer runs **three NLP checks in parallel** to prevent false alarms:

Check	Question Asked to Model	Threshold	Purpose
Hyperbole Detection	<i>"Is this exaggeration/venting or genuine distress?"</i>	$\geq 55\%$	Prevents <i>"I'm literally dying of boredom"</i> from being flagged as a crisis
Emergency Detection	<i>"Is this reporting a real emergency happening now?"</i>	$\geq 60\%$	Distinguishes <i>"There was a fire drill"</i> from <i>"The building is on fire"</i>
Target Detection	<i>"Is this threatening a specific person?"</i>	$\geq 55\%$	Separates <i>"I hate traffic"</i> from <i>"I know where he lives"</i>

Additionally, **keyword-based entity extraction** identifies specific emergency entities mentioned (e.g., *"ambulance"*, *"shooting"*, *"school bus"*) and location references (e.g., *"highway"*, *"hospital"*, *"bridge"*).

Output: `{ isHyperbole, hasEmergencyEntity, hasSpecificTarget, detectedEntities[], filterMethod }`

Phase 4 — Risk Engine & Severity Calculation

File: `ai-engine/filters/riskEngine.js`

A **10-rule deterministic engine** that combines outputs from Phases 1–3:

Rule	Condition	Result	Example
1	Emergency entity detected	Critical (4)	<i>"Explosion at the factory, people injured"</i>
2	Dominant emotion = joy/surprise/neutral	Low (1)	<i>"Amazing concert tonight!"</i>

3	Threat topic + hyperbole detected	Low (1)	<i>"My wife is so bad I want to kill someone"</i>
4	Civic topic + high negative emotion	Medium (2)	<i>"I'm furious, the road is broken for weeks"</i>
5	Self-Harm topic + despair/sadness ≥ 0.50	Critical (4)	<i>"I don't want to live anymore"</i>
6	Threat topic + high emotion + specific target	Critical (4)	<i>"I know where he lives, going tonight"</i>
7	Threat topic + high emotion + volatility flag	Critical (4)	fear: 0.90, anger: 0.88 — volatile
8	Threat topic + critical emotion ≥ 0.65	High (3)	<i>"I'm scared someone is following me"</i>
9	Threat topic + any high-risk emotion ≥ 0.65	High (3)	<i>"Disgusting crime in the neighborhood"</i>
10	Threat topic + moderate emotion	Medium (2)	<i>"Some bullying happening at school"</i>
Default	Everything else	Low (1)	General posts

Output: { severityLevel: "Critical"|"High"|"Medium"|"Low", severityScore: 1–4 }

6. Data Models (MongoDB Schemas)

6.1 User Model (`models/User.js`)

Field	Type	Details
name	String	Required
email	String	Required, unique, validated with regex
password	String	Required, min 5 chars, <code>select: false</code> (never returned)
role	String (Enum)	Only "admin" — single-role system
timestamps	Auto	createdAt, updatedAt

Security Features:

- Password hashing via bcryptjs (10 salt rounds) in a Mongoose `pre('save')` hook
- Password comparison via `matchPassword()` instance method
- Password field excluded from query results by default

6.2 Post Model (`models/Post.js`)

Section	Fields	Details
Raw Data	originalText, cleanedText, tokens[], source, uploaderName, uploaderLocation, postUrl	Source enum: twitter, reddit, facebook, manual_upload, instagram, dataset_upload
Topic (Engine A)	primaryTopic, topicConfidence	6-category enum, confidence 0.0–1.0
Emotions (Engine B)	emotions.{anger, fear, sadness, despair, disgust, joy, surprise, neutral}, dominantEmotion, dominantEmotionScore, volatilityFlag	All scores 0.0–1.0
Filters (Phase 3)	isHyperbole, hasEmergencyEntity, hasSpecificTarget, detectedEntities[]	Boolean flags + entity list
Severity (Phase 4)	severityLevel, severityScore	Enum: Low/Medium/High/Critical, Score: 1–4

Metadata	analysisMethod	nlp, partial_nlp, or keyword_fallback
Admin Review	reviewedByAdmin, adminNotes, isResolved	Manual review tracking
Timestamps	createdAt, updatedAt	Auto-generated

7. REST API Endpoints

7.1 Authentication (/api/auth)

Method	Endpoint	Description	Auth
POST	/api/auth/register	Register a new admin user (one-time setup)	No
POST	/api/auth/login	Login → JWT set in httpOnly cookie	No
POST	/api/auth/logout	Clear JWT cookie	Yes
GET	/api/auth/me	Get current user profile	Yes

7.2 Posts (/api/posts)

Method	Endpoint	Description	Auth
POST	/api/posts/analyze	Submit single post for AI analysis + save	Yes
GET	/api/posts	Get all posts (optional severity/topic filters)	Yes
GET	/api/posts/critical	Get Critical-severity posts only	Yes
PATCH	/api/posts/:id/resolve	Mark post resolved, add admin notes	Yes

7.3 Upload (/api/upload)

Method	Endpoint	Description	Auth
POST	/api/upload/dataset	Bulk upload (max 50 posts) for batch processing	Yes

7.4 Dashboard (/api/dashboard)

Method	Endpoint	Description	Auth
GET	/api/dashboard/stats	Aggregate statistics (severity, topics, emotions)	Yes

8. Frontend Architecture

8.1 Pages

Page	Route	Description
Login	/login	Admin authentication form
Dashboard	/dashboard	Aggregate metrics: severity distribution, topic/emotion breakdowns
Upload	/upload	Upload JSON/CSV datasets for bulk AI analysis
All Records	/records	Paginated incident list with severity/topic filters
Post Detail	/records/:id	Full AI analysis view + admin review panel

8.2 Components

Component	Purpose
Layout	Main application shell with sidebar navigation
Sidebar	Navigation menu (Dashboard, Upload, Records)
AuthContext	React Context API provider for global auth state

8.3 State Management

- **AuthContext** (React Context API) manages global authentication state
- On app mount, AuthContext calls `GET /api/auth/me` to check for an existing JWT cookie — enabling persistent sessions
- **ProtectedRoute** wrapper redirects unauthenticated users to `/login`

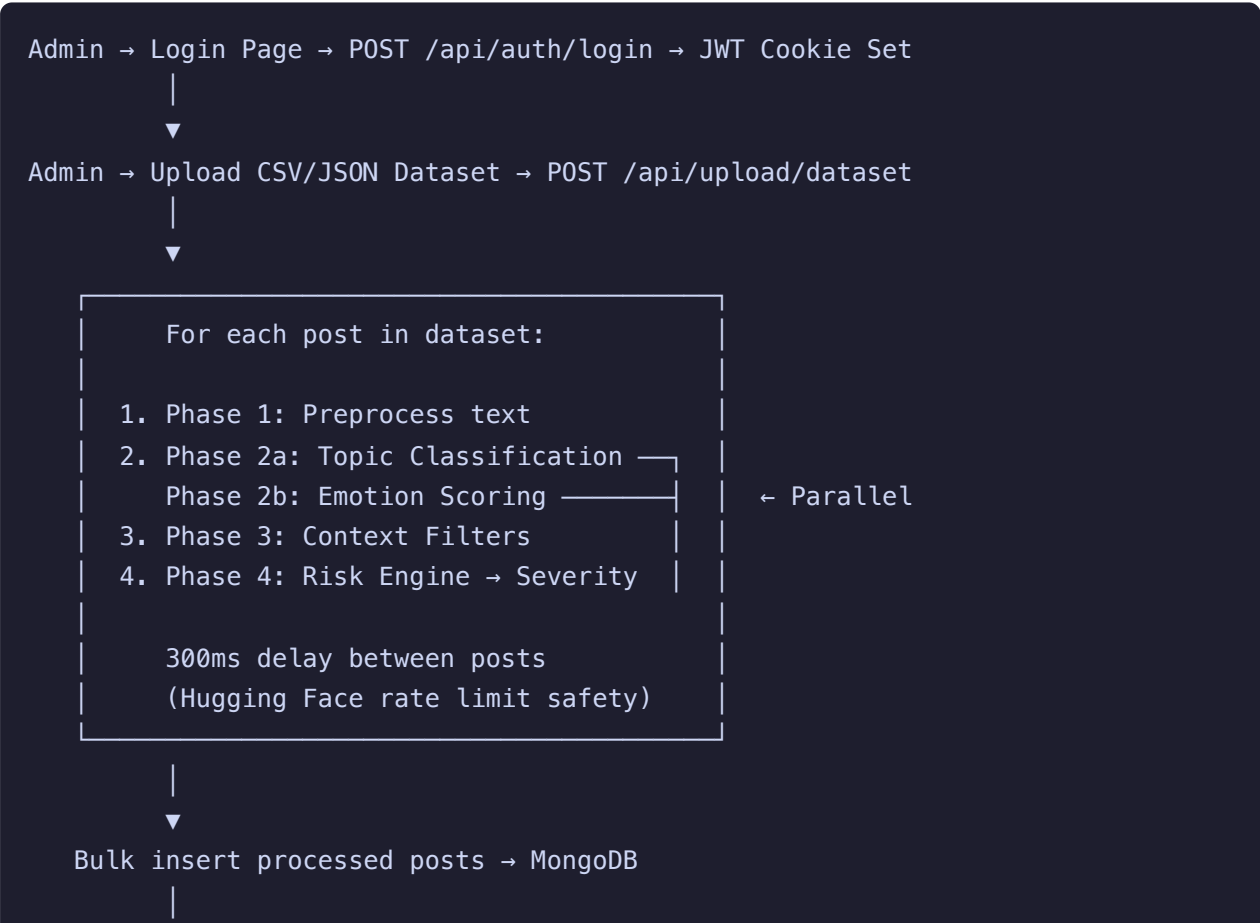
8.4 API Client (`api.js`)

Axios instance configured with `baseUrl: http://localhost:5001/api` and `withCredentials: true` for cookie-based authentication.

9. Authentication & Security

Security Layer	Implementation
Password Storage	bcrypt hashing (10 salt rounds), never stored in plaintext
Session Management	JWT tokens stored in httpOnly cookies (immune to XSS)
Cookie Security	sameSite: strict (CSRF protection), secure in production
Token Expiry	30-day expiration
HTTP Headers	Helmet.js (CSP, HSTS, X-Frame-Options, etc.)
CORS	Restricted to http://localhost:5173 with credentials
Route Protection	JWT middleware validates every protected endpoint
Input Validation	Express-Validator for request body validation
Error Handling	Centralized error handler prevents stack trace leakage

10. Data Flow — End-to-End User Journey



▼
Admin → Dashboard → GET /api/dashboard/stats
View severity counts, topic breakdown, emotion breakdown
|
▼
Admin → All Records → Filter by severity/topic
|
▼
Admin → Post Detail → Review AI verdict
→ Add admin notes → PATCH /api/posts/:id/resolve

11. Graceful Degradation Strategy

A key architectural decision is the **three-tier analysis method** tracking:

Method	Meaning	When It Happens
nlp	Both engines used Hugging Face AI	API available and responsive
partial_nlp	One engine used AI, other fell back to keywords	Partial API failure
keyword_fallback	Both engines used legacy keyword dictionaries	Complete API outage

This ensures the system **never crashes** due to external API unavailability. Every processed post stores its `analysisMethod` so admins can distinguish AI-verified results from keyword-approximated ones.

12. Key Design Decisions & Rationale

Decision	Rationale
Parallel AI calls (Promise.all)	Topic + Emotion engines are independent; parallel execution halves latency
Despair as a derived emotion	The HF model doesn't output "despair" directly; derived from sadness + context — critical for self-harm detection
Hyperbole filtering before severity	Without this, dramatic everyday language floods the Critical queue with false alarms
Sequential dataset processing	Respects Hugging Face Free Tier rate limits (300ms delay for batches > 10)
httpOnly JWT cookies	More secure than localStorage — immune to XSS attacks
Keyword fallback for every module	Ensures 100% uptime regardless of external API availability

Batch size limit (50 posts)	Prevents API timeouts and 429 rate-limit errors
Centralized orchestrator	analysisService.js decouples AI pipeline from controllers — easy to swap models

13. Project Structure

```

Mini Project/
├─ package.json                # Root concurrent dev scripts
├─ seed_data.js                # Database seeding script
├─ test_dataset.csv            # Sample CSV test data
├─ test_dataset.json           # Sample JSON test data
├─
├─ backend/backend/
│  ├─ server.js                # Express entry point
│  ├─ .env                     # Environment variables
│  ├─ models/
│  │  ├─ User.js               # Admin user schema
│  │  └─ Post.js               # Incident post schema (30+ fields)
│  ├─ controllers/
│  │  ├─ authController.js      # Register, Login, Logout, GetMe
│  │  ├─ postController.js      # CRUD + AI analysis
│  │  ├─ uploadController.js    # Bulk dataset processing
│  │  └─ dashboardController.js # Aggregate statistics
│  ├─ routes/
│  │  ├─ authRoutes.js
│  │  ├─ postRoutes.js
│  │  ├─ uploadRoutes.js
│  │  ├─ analysisRoutes.js
│  │  └─ dashboardRoutes.js
│  ├─ middleware/
│  │  ├─ authMiddleware.js      # JWT verification
│  │  └─ errorHandler.js       # Centralized error handling
│  ├─ services/
│  │  └─ analysisService.js     # Master AI pipeline orchestrator
│  └─ ai-engine/
│     ├─ engines/
│     │  ├─ topicClassifier.js  # BART-MNLI zero-shot
│     │  └─ emotionScorer.js    # DistilRoBERTa emotions
│     └─ filters/
│        └─ filterLayer.js      # Hyperbole/Emergency/Target

```

```

├── riskEngine.js           # 10-rule severity engine
├── utils/
│   └── preprocessor.js     # Text cleaning & tokenization
└── frontend/frontend/
    ├── index.html
    ├── vite.config.js
    └── src/
        ├── main.jsx        # Entry + AuthProvider + Router
        ├── App.jsx         # Route definitions
        ├── api.js          # Axios instance
        ├── index.css       # Global styles
        ├── context/
        │   └── AuthContext.jsx # Auth state (Context API)
        ├── components/
        │   ├── Layout/     # App shell with sidebar
        │   └── Sidebar/    # Navigation component
        └── pages/
            ├── Login/       # Admin login
            ├── Dashboard/   # Metrics & statistics
            ├── Upload/      # Dataset upload
            ├── AllRecords/  # Incident list
            └── PostDetail/  # Incident detail view

```

14. How to Run

1. **Start MongoDB** — must be running on port 27017
2. **Install dependencies:**
 - `cd backend/backend && npm install`
 - `cd frontend/frontend && npm install`
3. **Start both servers:** `npm run dev` (from project root)
4. **Register admin** (one-time): POST to `/api/auth/register` with name, email, password
5. **Open** `http://localhost:5173` and login

15. Future Scope

Enhancement	Description
-------------	-------------

Real-time social media ingestion	Integrate Twitter/X, Reddit, Facebook APIs for live monitoring
Multi-language support	Multilingual NLP models for Hindi, Spanish, etc.
Custom model fine-tuning	Train domain-specific models on labeled incident data
Role-based access control	Add viewer/moderator roles beyond admin-only
Geospatial mapping	Plot incidents on a map using location entities
Alert notifications	Email/SMS/push for Critical-severity incidents
Analytics & reporting	Time-series analysis, trend detection, exportable reports
Docker containerization	One-command deployment of entire stack

16. Summary

This project demonstrates a **production-grade architecture** for automated social media incident management. It combines the semantic understanding of transformer-based NLP models (BART, DistilRoBERTa) with a hand-crafted rules engine, wrapped in a secure full-stack web application. The system is designed to be **resilient** (graceful fallbacks), **fast** (parallel AI processing), and **accurate** (hyperbole filtering, derived emotions, multi-rule severity matrix) — making it a practical tool for civic authorities, NGOs, or platform moderators who need to rapidly identify and respond to real-world social issues at scale.